



OC Pizza

HandlePizza

Dossier d'exploitation

Version 0.1

Auteur

Antoine de Chaunac
Analyste programmeur



TABLE DES MATIERES

1 - Versions	4
2 - Introduction	5
2.1 - Objet du document	5
2.2 - Références.....	5
3 - Pré-requis d'exploitation	6
3.1 - Compte, accès et habilitations (AWS/GitHub/IAM)	6
3.2 - Environnements (DEV/TEST-UAT/PROD).....	6
3.3 - Services externes requis (bancaire, fournisseur).....	6
3.4 - Outils d'exploitation (CloudWatch, dashboards, runbook)	7
4 - Procédure de déploiement.....	8
4.1 - Déploiement backend (ECS Fargate).....	8
4.1.1 - Artefacts	8
4.1.2 - Variables d'environnement.....	8
4.1.3 - Configuration applicative	9
4.1.3.1 - Fichier application.yml.....	10
4.1.3.2 - Fichier application-<env>.yml	10
4.1.3.3 - Fichier logback-spring.xml.....	10
4.1.4 - Déploiement	10
4.1.5 - Vérifications post-déploiement	11
4.2 - Déploiement frontend (S3 + CloudFront + WAF).....	12
4.2.1 - Artefacts	12
4.2.2 - Publication des assets.....	12
4.2.3 - Invalidation cache CloudFront.....	12
4.2.4 - Vérification post-déploiement.....	13
4.3 - Migrations base de données (Flyway).....	13
4.3.1 - Pré-requis.....	13
4.3.2 - Exécution.....	13
4.3.3 - Controles.....	14
4.3.4 - Rollback / reprise	14
5 - Procédure de démarrage / arrêt.....	15
5.1 - Couche application (ECS).....	15
5.2 - Couche données (RDS, Redis, RabbitMQ).....	16
5.3 - Vérification de santé.....	16
6 - Procédure de mise à jour.....	18
6.1 - Mise à jour applicative.....	18
6.2 - Mise à jour base de données.....	18



6.3 - Mise à jour configuration / secrets.....	19
6.4 - Contrôles post-mise à jour.....	19
7 - Supervision et monitoring.....	21
7.1 - Logs centralisés (CloudWatch).....	21
7.2 - Métriques et alertes	21
7.3 - Supervision des integrations externes	22
7.4 - Gestion d'incident (runbook)	22
8 - Procédure de sauvegarde et restauration	24
8.1 - Politique de sauvegarde.....	24
8.2 - Procédure de restauration.....	24
8.3 - Tests périodiques de restauration	25
9 - Sécurité d'exploitation.....	26
9.1 - Gestion des secrets	26
9.2 - Gestion des accès IAM.....	26
9.3 - Chiffrement et conformité	27
10 - Glossaire	28



1 - VERSIONS

Auteur	Date	Description	Version
ADC	19/05/2026	Création du document	0.1
ADC	28/05/2026	Version 1 du document	1



2 - INTRODUCTION

2.1 - Objet du document

Le présent document constitue le dossier d'exploitation de l'application **HandlePizza** de la société **OC Pizza**.

Il décrit les conditions nécessaires à l'exploitation opérationnelle de la solution, les pré-requis techniques, les procédures de déploiement et de mise à jour, ainsi que les mécanismes de supervision, de sauvegarde et de reprise.

Ce document a pour objectif de :

- Formaliser les règles d'exploitation des environnements DEV, TEST/UAT et PROD ;
- Sécuriser les opérations techniques (déploiement, rollback, maintenance) ;
- Garantir la continuité de service et la fiabilité des traitements métier ;
- Fournir un référentiel commun aux équipes techniques, exploitation et support.

2.2 - Références

Pour de plus amples informations, se référer :

1. **DCT - Dossier_conception_technique** : dossier de conception technique de l'application
2. **DCF - Dossier_conception_fonctionnelle** : dossier de conception fonctionnelle de l'application



3 - PRÉ-REQUIS D'EXPLOITATION

3.1 - Compte, accès et habilitations (AWS/GitHub/IAM)

L'exploitation de **HandlePizza** nécessite les accès suivants :

- Compte AWS du projet avec séparation des droits par rôle (développement, exploitation, administration) ;
- Accès **GitHub** au repository et à la **pipeline GitHub Actions** ;
- **Habilitations IAM** appliquant le principe du moindre privilège ;
- Accès restreint aux secrets techniques via **AWS Secrets Manager** ;
- Traçabilité des accès et actions sensibles (audit).

Les accès de production sont limités aux profils autorisés et soumis à revue périodique.

3.2 - Environnements (DEV/TEST-UAT/PROD)

Trois environnements sont maintenus :

DEV : développement et validation technique rapide ;

TEST/UAT : recette technique et validation métier ;

PROD : exploitation en service réel.

Chaque environnement reprend la même architecture logique (exposition web, backend, données, supervision), avec un dimensionnement et des politiques de sécurité adaptés.

3.3 - Services externes requis (bancaire, fournisseur)

Les services externes nécessaires au fonctionnement sont :

- Système bancaire : autorisation/refus des transactions de paiement ;
- Plateforme fournisseur : échanges de réapprovisionnement et retours de réception.

Pré-requis :

DCAC
DCAC.COM

100 boulevard de Paris – 01 23 45 67 89 10 – DCACemail@dcac.com
S.A.R.L. au capital de 1 000,00 € enregistrée au RCS de Xxxx – SIREN 999 999 999 – Code APE : 6202A



- Endpoints accessibles depuis la couche applicative ;
- Certificats et mécanismes de sécurité valides ;
- Paramètres de timeout/retry/circuit breaker configurés ;
- Supervision des appels sortants activée.

3.4 - Outils d'exploitation (CloudWatch, dashboards, runbook)

L'exploitation s'appuie sur les outils suivants :

- **CloudWatch** : centralisation des logs, métriques et alertes ;
- **dashboards** : visualisation des indicateurs clés (latence, erreurs, charge, disponibilité) ;
- **runbook incident** : procédures de diagnostic, remédiation, escalade et communication.

Ces outils sont requis pour garantir une supervision proactive et une gestion maîtrisée des incidents.



4 - PROCÉDURE DE DÉPLOIEMENT

Cette section décrit les étapes opérationnelles de déploiement de **HandlePizza** sur les environnements cibles.

Elle couvre le déploiement backend et frontend, la gestion des configurations, les migrations de base de données et les contrôles post-déploiement, afin de garantir des mises en production fiables et traçables.

4.1 - Déploiement backend (ECS Fargate)

Le backend de **HandlePizza** est déployé sous forme de **services conteneurisés Spring Boot (Java 17)** exécutés sur **ECS Fargate**.

L'exposition des API est assurée via **ALB en HTTPS**, tandis que la configuration sensible est gérée hors code via **AWS Secrets Manager**.

4.1.1 - Artefacts

Le backend **HandlePizza** est livré sous forme d'image conteneurisée (**Spring Boot Java 17**), construite et versionnée par la **pipeline CI/CD**.

Contrairement à un modèle **batch ZIP**, le déploiement backend repose sur :

- Une image applicative versionnée (tag de release),
- Une définition de **service ECS Fargate**,
- Une configuration d'environnement externalisée,
- Des secrets techniques gérés hors code.

Artefacts principaux :

- Image backend **handlepizza-backend:<version>** ;
- Définition de **tâche ECS** (task definition) ;
- Paramètres d'environnement ;
- **Scripts SQL Flyway** versionnés (si migration DB associée).

4.1.2 - Variables d'environnement

DCAC	100 boulevard de Paris – 01 23 45 67 89 10 – DCACemail@dcac.com
DCAC.COM	S.A.R.L. au capital de 1 000,00 € enregistrée au RCS de Xxxx – SIREN 999 999 999 – Code APE : 6202A



Les variables d'environnement backend sont injectées au runtime par environnement (**DEV/TEST-UAT/PROD**) :

Nom	Obligatoire	Description
SPRING_PROFILES_ACTIVE	Oui	Profil Spring actif (dev, test, prod)
SERVER_PORT	Oui	Port applicatif du conteneur
DB_HOST	Oui	Endpoint base de données MySQL
DB_PORT	Oui	Port DB (généralement 3306)
DB_NAME	Oui	Nom du schéma applicatif
DB_USERNAME_SECRET_ARN	Oui	Référence secret login DB
DB_PASSWORD_SECRET_ARN	Oui	Référence secret mot de passe DB
REDIS_HOST	Oui	Endpoint Redis
MQ_HOST	Oui	Endpoint RabbitMQ / Amazon MQ
JWT_SECRET_ARN	Oui	Référence secret de signature JWT
BANK_API_BASE_URL	Oui	URL API système bancaire
SUPPLIER_API_BASE_URL	Oui	URL API fournisseur
LOG_LEVEL_ROOT	Non	Niveau de log global (INFO par défaut)

Les valeurs sensibles ne sont jamais stockées en clair dans le code ou dans les artefacts.

4.1.3 - Configuration applicative

La configuration est externalisée et pilotée par environnement.

Elle couvre notamment :

- **Paramètres Spring Boot ;**
- Configuration logs ;
- Paramètres de connexions DB/cache/messaging ;
- Paramètres d'intégration externe ;
- Paramètres de résilience (timeouts, retry, circuit breaker).



4.1.3.1 - Fichier *application.yml*

Fichier principal de configuration applicative (valeurs non sensibles), incluant :

- Paramètres serveur et profils ;
- Configuration des endpoints internes ;
- Réglages techniques généraux (pool, timeouts, etc.).

4.1.3.2 - Fichier *application-<env>.yml*

Surcharge spécifique par environnement (**application-dev.yml**, **application-test.yml**, **application-prod.yml**) :

- URLs de services ;
- Niveaux de logs ;
- Paramètres de performance adaptés au contexte.

4.1.3.3 - Fichier *logback-spring.xml*

Configuration de la journalisation :

- Format JSON ;
- Niveaux de logs ;
- Champs de corrélation ;
- Destination vers la collecte centralisée (**CloudWatch**).

4.1.4 - Déploiement

Le déploiement backend suit la séquence suivante :

- Sélectionner la version d'artefact à promouvoir ;
- Mettre à jour la **task definition ECS** (image + variables + secrets) ;
- Lancer le déploiement du **service ECS Fargate** ;
- Laisser l'**ALB router** progressivement vers les nouvelles tâches saines ;
- Exécuter les **migrations Flyway** si requises ;
- Valider le statut final du service (stable/running).

Le déploiement est automatisé via **GitHub Actions** et tracé dans l'historique **CI/CD**.



4.1.5 - Vérifications post-déploiement

Après déploiement, vérifier systématiquement :

- Etat healthy des tâches **ECS** ;
- Réussite des **health checks ALB** (/actuator/health) ;
- Accessibilité des **endpoints API** critiques ;
- Absence d'erreurs bloquantes dans **CloudWatch Logs** ;
- Latence et taux d'erreur dans les dashboards de supervision ;
- Connectivité aux dépendances (**RDS, Redis, MQ, services externes**).

En cas d'anomalie critique, appliquer la procédure de rollback définie en section 6.5.3.



4.2 - Déploiement frontend (S3 + CloudFront + WAF)

Le frontend de **HandlePizza** est déployé sous forme d'assets statiques (**HTML/CSS/JS**) publiés dans **S3** et distribués via **CloudFront**.

La protection en bordure est assurée par **WAF**.

4.2.1 - Artefacts

Les artefacts frontend sont produits par la **pipeline CI/CD** après build (**Vite**), puis versionnés.

Contenu type :

- index.html
- **Bundles JavaScript/CSS** optimisés
- Assets statiques (images, polices, icônes)

Chaque livraison frontend est associée à une version traçable (commit/tag/release).

4.2.2 - Publication des assets

La publication suit les étapes suivantes :

- Build du frontend sur la version ciblée ;
- Dépôt des assets dans le **bucket S3** de l'environnement cible ;
- Application des politiques de cache (longue durée pour assets fingerprintés, plus courte pour index.html) ;
- Vérification des droits d'accès et **métadonnées HTTP**.

Objectif : garantir une diffusion correcte des fichiers statiques sans rupture utilisateur.

4.2.3 - Invalidation cache CloudFront

Après publication, une invalidation **CloudFront** est exécutée pour forcer la prise en compte des nouvelles versions.

Règles usuelles :

- Invalidation ciblée (/index.html) au minimum ;

DCAC
DCAC.COM

100 boulevard de Paris – 01 23 45 67 89 10 – DCACemail@dcac.com
S.A.R.L. au capital de 1 000,00 € enregistrée au RCS de Xxxx – SIREN 999 999 999 – Code APE : 6202A



- Invalidation plus large (/*) en cas de changement majeur ;
- Traçabilité de l'invalidation dans la pipeline.

Objectif : éviter la persistance de contenus obsolètes côté edge.

4.2.4 - Vérification post-déploiement

Après déploiement frontend, contrôler :

- **Accessibilité HTTPS** de l'interface ;
- Chargement correct des assets (pas d'erreurs 404/403/5xx) ;
- Cohérence de version affichée ;
- Bon fonctionnement des parcours critiques ;
- Absence d'**alertes WAF** anormales.

En cas d'échec, revenir à la version frontend précédente (rollback artefact) et réexécuter les contrôles.

4.3 - Migrations base de données (Flyway)

Les évolutions de schéma de base de données sont gérées via **Flyway** et exécutées dans la chaîne de déploiement.

4.3.1 - Pré-requis

Avant exécution des migrations :

- Version applicative validée ;
- **Scripts Flyway** présents et versionnés ;
- **Accès DB** actif sur l'environnement cible ;
- **Sauvegarde DB** récente disponible ;
- Fenêtre de déploiement confirmée.

4.3.2 - Exécution

L'exécution suit la séquence suivante :

- Sélectionner l'environnement cible ;



- Exécuter **Flyway** sur les scripts non appliqués ;
- Vérifier le statut de migration (success/failure) ;
- Poursuivre le déploiement applicatif si la migration est validée.

Les scripts sont appliqués de manière ordonnée et traçable.

4.3.3 - Contrôles

Contrôles post-migration :

- Cohérence de la table d'historique **Flyway** ;
- Présence attendue des **nouveaux objets DB** ;
- Absence d'**erreurs SQL bloquantes** ;
- Validation fonctionnelle minimale sur parcours impactés.

Objectif : garantir la compatibilité entre schéma cible et version applicative déployée.

4.3.4 - Rollback / reprise

En cas d'échec migration :

- Interrompre la promotion vers l'environnement supérieur ;
- Appliquer la procédure de reprise documentée (correction script, relance contrôlée) ;
- Si nécessaire, restaurer la base depuis sauvegarde selon procédure de restauration ;
- Redéployer une version applicative compatible avec l'**état DB** restauré.

Le **rollback DB** est piloté avec prudence et toujours tracé dans les journaux d'exploitation.



5 - PROCÉDURE DE DÉMARRAGE / ARRÊT

Cette section décrit la procédure opérationnelle de démarrage et d'arrêt de **HandlePizza**, en distinguant la couche applicative, la couche données et les contrôles de santé associés.

L'objectif est de garantir des opérations maîtrisées, traçables et sans impact non contrôlé sur les utilisateurs.

5.1 - Couche application (ECS)

La couche applicative est déployée sur **ECS Fargate** et pilotée via les **services ECS**.

Démarrage :

- Vérifier la disponibilité des dépendances critiques (**RDS, Redis, RabbitMQ, Secrets Manager**).
- Vérifier la configuration cible (task definition, variables d'environnement, secrets).
- Positionner le **service ECS** sur le nombre de tâches souhaité (desired count > 0).
- Contrôler la montée progressive des tâches en état RUNNING puis HEALTHY.
- Vérifier la bonne prise en compte par l'**ALB** (targets saines).

Arrêt :

- Désactiver les déclenchements de déploiement automatique si nécessaire (fenêtre de maintenance).
- Réduire progressivement le desired count du **service ECS** jusqu'à 0.
- Vérifier le drainage du **trafic ALB** (plus de requêtes vers les tâches arrêtées).
- Contrôler l'absence d'erreurs anormales dans **CloudWatch Logs** pendant la phase d'arrêt.

Précaution :

- L'arrêt applicatif doit être réalisé hors pic d'activité et communiqué aux parties prenantes si impact utilisateur attendu.



5.2 - Couche données (RDS, Redis, RabbitMQ)

La couche de données est constituée de services managés en sous-réseau privé data.

Démarrage :

- Vérifier l'état Available de **RDS MySQL**.
- Vérifier la disponibilité d'**ElastiCache Redis**.
- Vérifier la disponibilité d'**Amazon MQ (RabbitMQ)** et des files/queues nécessaires.
- Confirmer la connectivité réseau depuis la couche applicative (security groups, routes, DNS internes).

Arrêt :

- En exploitation standard, les services data managés ne sont pas arrêtés systématiquement.
- En cas d'arrêt planifié exceptionnel :
 - o Arrêter d'abord la couche applicative (pour éviter les erreurs d'écriture/lecture).
 - o Appliquer les procédures spécifiques AWS du service concerné (RDS/Redis/MQ).
 - o Vérifier sauvegardes et points de restauration avant toute opération intrusive.

Précaution : Toute action d'arrêt sur la couche données doit être validée, tracée et couverte par une stratégie de reprise.

5.3 - Vérification de santé

Après tout démarrage (ou redémarrage), les contrôles suivants sont obligatoires :

Application :

- Endpoints de santé backend (/actuator/health) au statut attendu ;
- **Tâches ECS** en état stable ;
- Cibles **ALB healthy**.

Données et messaging :

- **Connexion DB** opérationnelle ;
- **Accès Redis** valide ;



- Publication/consommation RabbitMQ fonctionnelle.

Flux fonctionnels critiques :

- Appel API de test (authentification + endpoint métier simple) ;
- Vérification absence d'erreurs bloquantes dans CloudWatch Logs ;
- Contrôle métriques clés (latence, taux d'erreur, saturation ressources).

Critère de validation :

La plateforme est considérée opérationnelle lorsque les contrôles techniques et fonctionnels minimaux sont conformes, sans alerte critique active.



6 - PROCEDURE DE MISE A JOUR

Cette section décrit la procédure de mise à jour de **HandlePizza** pour les composants applicatifs, la base de données et la configuration technique.

L'objectif est de sécuriser les évolutions en limitant les risques de régression et en garantissant la traçabilité des changements.

6.1 - Mise à jour applicative

La mise à jour applicative couvre le backend (**ECS Fargate**) et le frontend (**S3/CloudFront**).

Séquence recommandée :

- Vérifier les prérequis de release (tests validés, artefacts versionnés, approbation de promotion).
- Déployer la nouvelle version backend via mise à jour de **task definition ECS**.
- Vérifier stabilité backend (tasks healthy, **health checks ALB**, logs sans erreur critique).
- Déployer les nouveaux assets frontend sur **S3**.
- Exécuter **l'invalidation CloudFront** (au minimum /index.html).
- Vérifier les parcours fonctionnels critiques.

Principes :

- Promotion d'artefacts immuables (pas de reconstruction entre environnements).
- Déploiement traçable via pipeline **CI/CD**.
- Rollback immédiat possible vers la version précédente en cas d'incident.

6.2 - Mise à jour base de données

Les évolutions de schéma sont versionnées et exécutées avec **Flyway**.

Séquence recommandée :

- Vérifier la présence des **scripts Flyway** versionnés et validés.
- Confirmer la disponibilité d'une **sauvegarde récente RDS**.
- Exécuter les migrations sur l'environnement cible.



- Contrôler **la table d'historique Flyway** et la cohérence des objets créés/modifiés.
- Valider la compatibilité applicative avec le schéma mis à jour.

Principes :

- Exécution ordonnée, non destructive et traçable.
- Fenêtre de déploiement maîtrisée pour les changements sensibles.
- Procédure de reprise documentée en cas d'échec de migration.

6.3 - Mise à jour configuration / secrets

La configuration et les secrets sont mis à jour sans modification du code source.

Configuration applicative :

- Mettre à jour les paramètres d'environnement (profil, endpoints, seuils, timeouts).
- Appliquer les changements dans les fichiers/profils concernés (application-<env>.yaml, paramètres runtime).
- Vérifier cohérence inter-environnements (**DEV/TEST-UAT/PROD**).

Secrets :

- Mettre à jour les secrets dans **AWS Secrets Manager (DB, JWT, clés API externes)**.
- Vérifier les **permissions IAM** des services consommateurs.
- Planifier/contrôler la rotation des secrets selon la politique sécurité.
- Tracer les changements (date, auteur, environnement, périmètre).

6.4 - Contrôles post-mise à jour

Après toute mise à jour, les contrôles suivants sont obligatoires :

Technique :

- **Services ECS** stables et healthy ;
- **Health checks ALB** valides ;
- **Connectivité RDS/Redis/RabbitMQ** opérationnelle ;
- Absence d'erreurs critiques dans **CloudWatch Logs**.

Fonctionnel :



- Vérification des parcours critiques (authentification, commande, paiement, suivi statut) ;
- Vérification des intégrations externes (banque/fournisseur) sur scénario nominal.

Supervision :

- Contrôle métriques clés (latence, taux d'erreur, saturation) ;
- Vérification absence d'alertes critiques post-déploiement ;
- Validation de la version effectivement active.

Critère de clôture :

La mise à jour est validée lorsque tous les contrôles techniques et fonctionnels sont conformes et qu'aucune alerte critique persistante n'est détectée.



7 - SUPERVISION ET MONITORING

Cette section décrit les mécanismes de supervision de **HandlePizza** pour garantir la disponibilité du service, détecter rapidement les anomalies et faciliter le diagnostic en exploitation.

7.1 - Logs centralisés (CloudWatch)

Les journaux applicatifs et techniques sont centralisés dans **CloudWatch Logs**.

Les applications publient des logs structurés (**format JSON**) avec horodatage, niveau de sévérité et identifiants de corrélation.

Contrôles d'exploitation recommandés :

- Vérifier la présence continue des flux de logs backend ;
- Vérifier la politique de rétention selon environnement (**DEV/TEST-UAT/PROD**).
- Contrôler l'absence de pics **d'erreurs ERROR/WARN** anormaux ;
- Rechercher les exceptions critiques sur les parcours métier sensibles ;
- Corréler les logs avec l'heure des incidents remontés ;

Vérifier la politique de rétention selon environnement (**DEV/TEST-UAT/PROD**).

Objectif : disposer d'une base fiable pour diagnostic, audit et post-mortem incident.

7.2 - Métriques et alertes

La supervision métrique couvre les indicateurs clés de performance et de stabilité de la plateforme.

Métriques principales :

- Disponibilité des **services ECS/ALB** ;
- **Latence API (p50/p95/p99** si disponible) ;
- **Taux d'erreurs HTTP** (4xx/5xx) ;
- CPU/mémoire des tâches backend ;
- Saturation des **connexions DB / latence RDS** ;
- État des **files MQ** et consommation.



Alerting :

- Seuils d'alerte définis par criticité (warning/critical) ;
- Notification vers les canaux d'astreinte/exploitation ;
- Escalade automatique si incident non résolu dans le délai attendu.

Objectif : détecter proactivement les dérives avant impact utilisateur majeur.

7.3 - Supervision des integrations externes

Les appels vers les services bancaires et fournisseur sont supervisés en continu.

Indicateurs suivis :

- Latence des appels sortants ;
- Taux d'échec et timeouts ;
- Volume de retries ;
- Fréquence d'ouverture des circuit breakers ;
- Disponibilité perçue des partenaires externes.

Contrôles :

- Vérifier la cohérence entre erreurs techniques et statuts métier ;
- Déclencher alerte en cas de dégradation persistante ;
- Isoler rapidement un flux externe défaillant pour préserver le cœur applicatif.

Objectif : maintenir un service stable malgré la dépendance à des systèmes tiers.

7.4 - Gestion d'incident (runbook)

La gestion d'incident est pilotée via un **runbook opérationnel**.

Étapes minimales :

- Qualification (impact, criticité, périmètre) ;
- Diagnostic initial (logs, métriques, traces, dépendances) ;
- Actions de remédiation standard (redémarrage, rollback, limitation de flux) ;
- Escalade vers les rôles techniques/métier ;
- Communication de statut jusqu'à résolution ;



- Clôture avec retour d'expérience (**RCA et actions préventives**).

Objectif : réduire **le MTTR** et homogénéiser la réponse opérationnelle.



8 - PROCÉDURE DE SAUVEGARDE ET RESTAURATION

Cette section formalise la stratégie de protection des données, de restauration et de vérification périodique pour garantir la continuité d'activité.

8.1 - Politique de sauvegarde

La politique de sauvegarde couvre en priorité les données métier critiques (**RDS MySQL**).

Principes retenus :

- Sauvegardes automatiques planifiées ;
- Politique de rétention définie par environnement ;
- Chiffrement des sauvegardes au repos ;
- Traçabilité des opérations de sauvegarde ;
- Vérification régulière de la complétude des sauvegardes.

Prérequis organisationnels :

- Définir **RPO/RTO** cibles ;
- Documenter fenêtres de sauvegarde et responsabilités d'exploitation.

8.2 - Procédure de restauration

En cas d'incident de données, la restauration suit une procédure contrôlée.

Séquence recommandée :

- Qualifier l'incident et décider la stratégie de restauration ;
- Identifier le point de restauration cible ;
- Restaurer la base sur instance cible (ou instance temporaire de validation) ;
- Vérifier l'intégrité des données restaurées ;
- Reconnecter l'application de manière contrôlée ;
- Exécuter les contrôles techniques et fonctionnels post-restauration ;
- Tracer l'opération (date, version, opérateur, périmètre).



Précaution : toute restauration en production doit être validée et communiquée avant exécution.

8.3 - Tests périodiques de restauration

Les restaurations doivent être testées régulièrement pour valider l'opérabilité réelle du dispositif de secours.

Principes retenus :

- Planification de tests de restauration à fréquence définie ;
- Exécution sur environnement contrôlé ;
- Mesure des temps de reprise (**RTO réel**) ;
- Vérification de cohérence fonctionnelle des données restaurées ;
- Capitalisation des résultats et actions d'amélioration.

Objectif : garantir que la restauration est réellement exécutable dans les délais attendus en cas de crise.



9 - SÉCURITÉ D'EXPLOITATION

Cette section décrit les règles de sécurité opérationnelle appliquées à **HandlePizza** en exploitation.

Elle couvre la protection des secrets, le contrôle des accès aux ressources cloud et les mécanismes de chiffrement/conformité nécessaires au maintien d'un niveau de sécurité adapté en production.

9.1 - Gestion des secrets

Les secrets techniques (**identifiants DB, clés API externes, secrets JWT, tokens techniques**) sont gérés exclusivement hors code via **AWS Secrets Manager**.

Principes retenus :

- Aucun secret en clair dans le code source, les artefacts, les pipelines ou les fichiers de configuration versionnés ;
- Récupération des secrets au runtime par les services autorisés ;
- Rotation planifiée des secrets selon la politique de sécurité ;
- Journalisation et audit des accès/modifications de secrets ;
- Révocation/remplacement immédiat en cas de suspicion de compromission.

Contrôles d'exploitation :

- Vérifier régulièrement la date de dernière rotation ;
- Vérifier la portée IAM des rôles consommateurs ;
- Vérifier l'absence d'exposition de secrets dans les logs applicatifs.

9.2 - Gestion des accès IAM

Le contrôle d'accès aux **ressources AWS** s'appuie sur **IAM (Identity and Access Management)** avec application stricte du principe de moindre privilège.

Principes retenus :

- Attribution de permissions minimales par rôle technique (application, **CI/CD**, exploitation, administration) ;
- Séparation des rôles et responsabilités (pas de compte partagé privilégié non tracé) ;



- Limitation des accès humains directs aux environnements sensibles, en particulier PROD ;
- Revue périodique des droits et suppression des accès obsolètes ;
- Traçabilité des actions d'administration et d'exploitation.

Contrôles d'exploitation :

- Vérifier les **politiques IAM** critiques ;
- Contrôler les élévations de privilèges ;
- Auditer les accès à fréquence définie.

9.3 - Chiffrement et conformité

La protection des données est assurée en transit et au repos, avec des pratiques de conformité alignées sur les exigences de sécurité internes.

Chiffrement en transit :

- **HTTPS/TLS** pour les flux clients, API et intégrations externes ;
- Canaux sécurisés entre composants internes selon les capacités des services managés.

Chiffrement au repos :

- Chiffrement des données stockées (**RDS**, sauvegardes, **S3**, services de cache/messaging lorsque supporté) ;
- Protection des snapshots/sauvegardes selon la même politique.

Conformité opérationnelle :

- Masquage des données sensibles dans les logs ;
- Conservation maîtrisée des traces et journaux ;
- Application des règles de confidentialité sur les environnements non production (anonymisation/pseudonymisation des données de test).

Objectif : garantir confidentialité, intégrité et traçabilité des données sur l'ensemble du cycle d'exploitation.



10 - GLOSSAIRE

ACM (AWS Certificate Manager)	Service AWS de gestion des certificats TLS (création, renouvellement, association aux services d'exposition).
ALB (Application Load Balancer)	Equilibreur de charge applicative AWS distribuant le trafic HTTP / HTTPS vers les services backend disponibles.
AMQP (Advanced Message Queuing Protocol)	Protocole d'échange de messages utilisé par RabbitMQ/Amazon MQ.
API (Application Programming Interface)	Interface permettant à deux applications de communiquer via des contracts techniques définis.
Artefact	Résultat d'un build (image backend, bundle frontend, etc.) prêt à être déployé.
Audit trail	Historique horodaté des actions critiques (qui a fait quoi, quand, où).
Auto scaling	Mécanisme d'ajustement automatique de capacité selon la charge (CPU, mémoire, trafic, etc.)
AWS (Amazon Web Service)	Plateforme cloud utilisée pour héberger et exploiter handlePizza.
Backend	Partie serveur de l'application, porteuse des règles métier et des APIs.
Bean validation	Standard Java de validation des données entrantes (formats, contraintes, champs obligatoires).
Cache	Mécanisme de stockage temporaire en mémoire pour accélérer les lectures fréquentes.
CI/CD (Continuous integration / continuous delivery)	Chaine d'automatisation de build, tests, contrôles, déploiement et promotion des versions.
CloudFront	CDN AWS de distribution de contenus statiques avec cache en edge.
CloudWatch	Service AWS de supervision (logs, métriques, alertes, tableaux de bord).
Contrat API	Définition formelle d'une API : endpoints, méthodes, schémas, statuts,



	erreurs.
Controller	Couche backend exposant les endpoints REST et recevant les requêtes HTTP.
DTO (Data transfer object)	Objet d'échange API distinct des entités de persistance.
ECS Fargate	Service AWS d'exécution de conteneurs sans gestion direct de serveurs.
Edge web	Couche d'entrée internet recevant, filtrant, sécurisant et routant les flux vers les services internes.
ElastiCache Redis	Service AWS managé Redis, utilisé pour le cache applicatif.
Endpoint	URL d'accès à une opération API.
Environnement (DEV / TEST-UAT / PROD)	Contexte d'exécution applicatif : développement, recette, production.
Feature flag	Paramètre d'activation contrôlée d'une fonctionnalité sans redéploiement complet.
Flyway	Outil de versionnement et d'exécution des migrations de schéma de base de données.
Frontend (SPA)	Partie interface utilisateur web : ici application monopage React/TypeScript.
GitHub Actions	Service d'automatisation CI/CD intégré à GitHub.
Health check	Vérification automatique de l'état d'un service (ex. endpoint /actuator/health).
HTTPS	Protocole HTTP chiffré via TLS.
IAM (Identity and Access Management)	Service AWS de gestion des identités, rôles et permissions d'accès.
Idempotence	Propriété garantissant qu'une même opération répétée produit un résultat cohérent sans effets de bord supplémentaire.
JDBC (Java Database Connectivity)	Interface Java de connexion et d'accès aux bases relationnelles.
JPA (Java)	Spécification Java d'accès/persistance objet-relationnelles.



Persistence API)	
JWT (JSON Web Token)	Jeton signé transportant l'identité et les droits d'un utilisateur pour l'authentification API.
Latence	Temps de réponse d'un service entre requête et réponse.
Logback	Framework de journalisation Java (ici utilisé en format JSON structuré).
MapStruct	Outil de génération de mapping entre DTO et objets métier/entités.
Messaging	Echanges asynchrones entre composants via broker de messages.
Micrometer	Bibliothèque de collecte de métriques applicatives.
Monolithe modulaire	Application déployée en une unité, mais organisée en modules métier séparés.
MQ (Message Queue)	Mécanisme/file de messages pour traitements asynchrones et découplés.
MySQL 8 / RDS MySQL	Base relationnelle (service managé AWS via RDS) stockant les données métier.
Nat Gateway	Passerelle de sortie internet pour ressources privées, sans exposition entrante directe.
OpenAPI	Spécification standard de description des APIs REST.
RBAC (Role-Based Access Control)	Autorisation par rôles (client, employé, manager, direction, etc.) et périmètre.
RDS (Relationnal Database Service)	Service AWS manage pour bases relationnelles.
Redis	Base clé-valeur en mémoire utilisée pour le cache et l'accélération des accès.
Repository	Couche backend responsable des opérations d'accès aux données.
Resilience4j	Bibliothèque de résilience applicative (timeouts, retry, circuit breaker, bulkhead).
REST (Representational State Transfer)	Style d'architecture API base sur les ressources HTTP et verbes standards.
Retry	Nouvelle tentative contrôlée après échec transitoire.
Rollback	Retour à une version précédente stable après incident de



	déploiement/mise à jour.
RPO (Recovery Point Objective)	Perte de données maximale acceptable (objectif de point de reprise).
RTO (Recovery Time Objective)	Durée maximale acceptable de reprise de service.
Runbook	Procédure opérationnelle documentée de gestion d'incident/exploitation.
S3 (Simple Storage Service)	Service AWS de stockage objet, utilisé pour les assets frontend.
Secrets Manager	Service AWS de gestion sécurisée des secrets techniques (DB, JWT, API keys).
Service	Couche backend contenant la logique métier.
SLA / SLO	Objectifs d'engagement et de niveau de service (disponibilité, latence, etc.).
Spring Boot	Framework Java pour construire des applications backend et APIs rapidement.
Spring Security	Framework de sécurité backend (authentification, autorisation, filtres).
Swagger UI	Interface web générée depuis OPENAPI pour explorer et tester les endpoints.
Task Definition (ECS)	Définition technique du conteneur à exécuter (image, ressources, variables, secrets).
TLS (Transport Layer Security)	Protocole de chiffrement des communications réseau (base de HTTPS).
Timeout	Durée maximale d'attente d'une réponse avant échec contrôlé.
VPC (Virtual Private Cloud)	Réseau privé isolé dans AWS, segmenté en sous-réseaux publics/privés.
WAF (Web Application Firewall)	Pare-feu applicative web filtrant les requêtes HTTP/HTTPS malveillantes.